Course Title : Web Technologies
Course Code : DI 322
BS.IT 5TH Semester
Document Made by: Miss Areej Ali

Topic 8: CSS3

TABLE OF CONTENTS

CSS3 | Introduction

CSS stands for Cascading Style Sheets. CSS is a standard style sheet language used for describing the presentation (i.e. the layout and formatting) of the web pages.
Prior to CSS, nearly all of the presentational attributes of HTML documents were contained within the HTML markup (specifically inside the HTML tags); all the font colors, background styles, element alignments, borders and sizes had to be explicitly described within the HTML. As a result, development of the large websites became a long and expensive process, since the style information was repeatedly added to every single page of the website.
To solve this problem CSS was introduced in 1996 by the World Wide Web Consortium (W3C), which also maintains its standard. CSS was designed to enable the separation of presentation and content. Now web designers can move the formatting information of the web pages to a separate style sheet which results in considerably simpler HTML markup, and better maintainability.
CSS3 is the latest version of the CSS specification. CSS3 adds several new styling features and improvements to enhance the web presentation capabilities.

Getting Started with CSS3

1. CSS3 Border

The CSS3 provides two new properties for styling the borders of an element in a more elegant way — the border-image property for adding the images to borders, and the border-radius property for making the rounded corners without using any images.

- **Creating CSS3 Rounded Corners**

The border-radius property can be used to create rounded corners. This property typically defines the shape of the corner of the outer border edge. Prior to CSS3, sliced images are used for creating the rounded corners that was rather bothersome.

```
.box

{ width: 300px;

 height: 150px;

 background: #ffb6c1;
```

```
border: 2px solid #f08080;

border-radius: 20px; }
```

• Adding CSS3 Border Images

The border-image property allows you to specify an image to act as an element's border. The design of the border is created from the sides and corners of the image specified in border-image source URL. The border image may be sliced, repeated, scaled and stretched in various ways to fit the size of the border image area.

```
.box

{ width: 300px;

 height: 150px;

border: 15px solid transparent;

 -webkit-border-image: url("border.png") 30 30 round; /* Safari 3.1-5 */

-o-border-image: url("border.png") 30 30 round; /* Opera 11-12.1 */

border-image: url("border.png") 30 30 round; }
```

Tip: The round option is a variation of the repeat option that distributes the image tiles in such a way that the ends are nicely connected.

2. CSS3 Color

In addition to that CSS3 adds some new functional notations for setting color values for the elements which are rgba(), hsl() and hsla().
In the following section we'll discuss about these color model one by one.

RGBA Color Values

Colors can be defined in the RGBA model (red-green-blue-alpha) using the rgba() functional notation. RGBA color model are an extension of RGB color model with an alpha channel — which specifies the opacity of a color.
The alpha parameter accepts a value from 0.0 (fully transparent) to 1.0 (fully opaque).

```
h1 { color: rgba(0,0,255,0.5); }

p { background-color: rgba(0%,0%,100%,0.3); }
```

HSL Color Values

Colors also can be defined the HSL model (hue-saturation-lightness) using the hsl() functional notation. Hue is represented as an angle (from 0 to 360) of the color wheel or circle (i.e. the rainbow represented in a circle). This angle is given as a unit less number because the angle is so typically measured in degrees that the unit is implicit in CSS.
Saturation and lightness are represented as percentages. 100% saturation means full color, and 0% is a shade of gray. Whereas, 100% lightness is white, 0% lightness is black, and 50% lightness is normal. Check out the example below:

```
h1 { color: hsl(360,70%,60%); } p { background-color: hsl(480,50%,80%); }
```

Tip: By the definition red=0=360, and the other colors are spread around the circle, so green=120, blue=240, etc. As an angle, it implicitly wraps around such that -120=240, 480=120, and so on.

HSLA Color Values

Colors can be defined in the HSLA model (hue-saturation-lightness-alpha) using the hsla() functional notation. HSLA color model are an extension of HSL color model with an alpha channel — which specifies the opacity of a color.
The alpha parameter accepts a value from 0.0 (fully transparent) to 1.0 (fully opaque).

```
h1 { color: hsla(360,80%,50%,0.5); } p { background-color: hsla(480,60%,30%,0.3); }
```

3. CSS3 Background

The CSS3 provides several new properties to manipulate the background of an element like background clipping, multiple backgrounds, and the option to adjust the background size.
The following section will describe you all the new background features of CSS3, for other background related properties please check out the CSS background.

• CSS3 background-size Property

The background-size property can be used to specify the size of the background images. Prior to CSS3, the size of the background images was determined by the actual size of the images. The background image size can be specified using the pixels or percentage values as well as the keywords auto, contain, and cover. Negative values are not allowed.

```
.box { width: 250px; height: 150px; background: url("images/sky.jpg") no-repeat;
background-size: contain; border: 6px solid #333; }
```

Tip: The background-size property is typically used to create full size background images that scale according to the size of viewport or witdh of the browser.

• CSS3 background-clip Property

The background-clip property can be used to specify whether an element's background extends into the border or not. The background-clip property can take the three values: border-box, padding-box, content-box.

```
.box

 { width: 250px;

height: 150px; padding: 10px;

border: 6px dashed #333;

background: orange;
```

```
background-clip: content-box; }
```

CSS3 background-origin Property

The background-origin property can be used to specify the positioning area of the background images. It can take the same values as background-clip property: border-box, padding-box, content-box.

```
.box

{ width: 250px;

 height: 150px;

padding: 10px;

border: 6px dashed #333;

background: url("images/sky.jpg") no-repeat;

background-size: contain;

background-origin: content-box; }
```

Note: The background-origin property is ignored if the value of background-attachment property is specified as 'fixed'.

CSS3 Multiple Backgrounds

CSS3 gives you the ability to add multiple backgrounds to a single element. The backgrounds are layered on the top of one another. The number of layers is determined by the number of comma-separated values in the background-image or background shorthand property.

```
.box { width: 100%; height: 500px; background: url("images/birds.png") no-repeat center, url("images/clouds.png") no-repeat center, lightblue; }
```

The first value in the comma-separated list of backgrounds i.e. the background-image 'birds.png' will appear on the top and the last value i.e. the 'light blue' color will appear at the bottom. Only the last background can include a background-colour.

4. CSS3 Gradients

The CSS3 gradient feature provides a flexible solution to generate smooth transitions between two or more colors. Earlier, to achieve such effect we had to use the images. Using CSS3 gradients you can reduce the download time and saves the bandwidth usages. The elements with gradients can be scaled up or down to any extent without losing the quality, also the output will render much faster because it is generated by the browser.
Gradients are available in two styles: linear and radial.

• Creating CSS3 Linear Gradients

To create a linear gradient you must define at least two color stops. However to create more complex gradient effects you can define more color stops. Color stops are the colors you want to render smooth transitions among. You can also set a starting point and a direction (or an angle) along which the gradient effect is applied. The basic syntax of creating the linear gradients using the keywords can be given with:

```
linear-gradient(direction, color-stop1, color-stop2, ...)
```

• Linear Gradient - Top to Bottom

The following example will create a linear gradient from top to bottom. This is default.

```
.gradient

{ /* Fallback for browsers that don't support gradients */

background: red; /* For Safari 5.1 to 6.0 */

background: -webkit-linear-gradient(red, yellow); /* For Internet Explorer 10 */
background: -ms-linear-gradient(red, yellow); /* Standard syntax */

background: linear-gradient(red, yellow); }
```

- ## Linear Gradient - Left to Right

The following example will create a linear gradient from left to right.

```
.gradient

{ /* Fallback for browsers that don't support gradients */

background: red; /* For Safari 5.1 to 6.0 */

background: -webkit-linear-gradient(left, red, yellow); /* For Internet Explorer 10 */
background: -ms-linear-gradient(left, red, yellow); /* Standard syntax */

background: linear-gradient(to right, red, yellow); }
```

- ## Linear Gradient - Diagonal

You can also create a gradient along the diagonal direction. The following example will create a linear gradient from the bottom left corner to the top right corner of the element's box.

```
.gradient

{ /* Fallback for browsers that don't support gradients */

background: red; /* For Safari 5.1 to 6.0 */

background: -webkit-linear-gradient(bottom left, red, yellow); /* For Internet Explorer 10 */

background: -ms-linear-gradient(bottom left, red, yellow); /* Standard syntax */
background: linear-gradient(to top right, red, yellow); }
```

- ## Setting Direction of Linear Gradients Using Angles

If you want more control over the direction of the gradient, you can set the angle, instead of the predefined keywords. The angle 0deg creates a bottom to top gradient, and positive angles represent clockwise rotation, that means the angle 90deg creates a left to right gradient. The basic syntax of creating the linear gradients using angle can be given with:

```
linear-gradient(angle, color-stop1, color-stop2, ...)
```

The following example will create a linear gradient from left to right using angle.

```
.gradient

{ /* Fallback for browsers that don't support gradients */ background: red; /* For Safari 5.1 to 6.0 */

background: -webkit-linear-gradient(0deg, red, yellow); /* For Internet Explorer 10 */

background: -ms-linear-gradient(0deg, red, yellow); /* Standard syntax */

background: linear-gradient(90deg, red, yellow); }
```

• Creating Linear Gradients Using Multiple Color Stops

You can also create gradients for more than two colors. The following example will show you how to create a linear gradient using multiple color stops. All colors are evenly spaced.

```
.gradient

{ /* Fallback for browsers that don't support gradients */

background: red; /* For Safari 5.1 to 6.0 */

background: -webkit-linear-gradient(red, yellow, lime); /* For Internet Explorer 10 */
background: -ms-linear-gradient(red, yellow, lime); /* Standard syntax */

background: linear-gradient(red, yellow, lime); }
```

• Setting the Location Color Stops

Color stops are points along the gradient line that will have a specific color at that location. The location of a color stop can be specified either as a percentage, or as an absolute length. You may specify as many color stops as you like to achieve the desired effect.

```
.gradient
```

```
{ /* Fallback for browsers that don't support gradients */

background: red; /* For Safari 5.1 to 6.0 */

background: -webkit-linear-gradient(red, yellow 30%, lime 60%); /* For Internet Explorer 10 */

background: -ms-linear-gradient(red, yellow 30%, lime 60%); /* Standard syntax */
background: linear-gradient(red, yellow 30%, lime 60%); }
```

Note: While setting the color-stops location as a percentage, 0% represents the starting point, while 100% represents the ending point. However, you can use values outside of that range i.e. before 0% or after 100% to get the effect you want.

• Repeating the Linear Gradients

You can repeat linear gradients using the repeating-linear-gradient() function.

```
.gradient { /* Fallback for browsers that don't support gradients */

 background: white; /* For Safari 5.1 to 6.0 */

 background: -webkit-repeating-linear-gradient(black, white 10%, lime 20%); /* For Internet Explorer 10 */

 background: -ms-repeating-linear-gradient(black, white 10%, lime 20%); /* Standard syntax */

 background: repeating-linear-gradient(black, white 10%, lime 20%); }
```

Creating CSS3 Radial Gradients

In a radial gradient color emerge from a single point and smoothly spread outward in a circular or elliptical shape rather than fading from one color to another in a single direction as with linear gradients. The basic syntax of creating a radial gradient can be given with:

```
radial-gradient(shape size at position, color-stop1, color-stop2, ...);
```

The arguments of the radial-gradient() function has the following meaning:

- position — Specifies the starting point of the gradient, which can be specified in units (px, em, or percentages) or keyword (left, bottom, etc).
- shape — Specifies the shape of the gradient's ending shape. It can be circle or ellipse.
- size — Specifies the size of the gradient's ending shape. The default is farthest-side.

The following example will show you create a radial gradient with evenly spaced color stops.

```
.gradient

{ /* Fallback for browsers that don't support gradients */

background: red; /* For Safari 5.1 to 6.0 */

background: -webkit-radial-gradient(red, yellow, lime); /* For Internet Explorer 10 */
background: -ms-radial-gradient(red, yellow, lime); /* Standard syntax */

 background: radial-gradient(red, yellow, lime); }
```

Setting the Shape of Radial Gradients

The shape argument of the radial-gradient() function is used to define the ending shape of the radial gradient. It can take the value circle or ellipse. Here's is an example:

```
.gradient { /* Fallback for browsers that don't support gradients */ background: red; /*
For Safari 5.1 to 6.0 */ background: -webkit-radial-gradient(circle, red, yellow, lime); /*
For Internet Explorer 10 */ background: -ms-radial-gradient(circle, red, yellow, lime);
/* Standard syntax */ background: radial-gradient(circle, red, yellow, lime); }
```

Note: If the shape argument is omitted or not specified, the ending shape defaults to a circle if the size is a single length, otherwise an ellipse.

Setting the Size of Radial Gradients

The size argument of the radial-gradient() function is used to define the size of the gradient's ending shape. Size can be set using units or the keywords: closest-side, farthest-side, closest-corner, farthest-corner.

```
.gradient {

 /* Fallback for browsers that don't support gradients */

background: red; /* For Safari 5.1 to 6.0 */

background: -webkit-radial-gradient(left bottom, circle farthest-side, red, yellow, lime);
/* For Internet Explorer 10 */

background: -ms-radial-gradient(left bottom, circle farthest-side, red, yellow, lime); /*
Standard syntax */

 background: radial-gradient(circle farthest-side at left bottom, red, yellow, lime); }
```

Repeating the Radial Gradients

You can also repeat radial gradients using the repeating-radial-gradient() function.

```
.gradient

{ /* Fallback for browsers that don't support gradients */ background: white; /* For
Safari 5.1 to 6.0 */

background: -webkit-repeating-radial-gradient(black, white 10%, lime 20%); /* For
Internet Explorer 10 */

 background: -ms-repeating-radial-gradient(black, white 10%, lime 20%); /* Standard
syntax */
```

```
background: repeating-radial-gradient(black, white 10%, lime 20%); }
```

• CSS3 Transparency and Gradients

CSS3 gradients also support transparency. You can use this to create fading effects on background images when stacking multiple backgrounds.

```
.gradient

{ /* Fallback for browsers that don't support gradients */

 background: url("images/sky.jpg"); /* For Safari 5.1 to 6.0 */

background: -webkit-linear-gradient(left, rgba(255,255,255,0), rgba(255,255,255,1)),
url("images/sky.jpg"); /* For Internet Explorer 10 */

background: -ms-linear-gradient(left, rgba(255,255,255,0), rgba(255,255,255,1)),
url("images/sky.jpg"); /* Standard syntax */

background: linear-gradient(to right, rgba(255,255,255,0), rgba(255,255,255,1)),
url("images/sky.jpg"); }
```

5. CSS3 Text Overflow

Handling Text Overflow in CSS3

CSS3 introduced several new property properties for modifing the text contents, however some of these properties are existed from a long time. These properties give you precise control over the rendering of text on the web browser.

• Hiding Overflow Text

Text can overflow, when it is prevented from wrapping, for example, if the value of white-space property is set to nowrap for the containing element or a single word is too long to fit like a long email address. In such situation you can use the CSS3 text-overflow property to determine how the overflowed text content will be displayed.
You can either display or clip the overflowed text or clip and display an ellipsis or a custom string in palace of the clipped text to indicate the user.
Values Accepted by the word-break property are: clip and ellipsis and string.

```
p { width: 400px;

overflow: hidden;

white-space: nowrap;

 background: #cdcdcd; }

p.clipped

{ text-overflow: clip; /* clipped the overflowed text */ }

 p.ellipses { text-overflow: ellipsis; /* display '…' to represent clipped text */ }
```

Warning: The string value for the text-overflow property is not supported in most of the web browsers, you should better avoid this.

• Breaking Overflow Text

You can also break a long word and force it to wrap onto a new line that overflows the boundaries of the containing element using the CSS3 word-wrap property.
Values Accepted by the word-wrap property are: normal and break-word.

```
p { width: 200px; background: #ffc4ff; word-wrap: break-word; }
```

Tip: Please check out the individual property reference for all the possible values and the Browser support for these CSS properties.

• Specify Word Breaking Rules

You can also specify the line breaking rules for the text (i.e. how to break lines within words) yourself using the CSS3 word-break property.
Values Accepted by the word-break property are: normal, break-all and keep-all.

```
p { width: 150px; padding: 10px; }

p.break { background: #bedb8b; word-break: break-all; }
```

```
p.keep { background: #f09bbb; word-break: keep-all; }
```

6. CSS3 Drop Shadows

The CSS3 gives you ability to add drop shadow effects to the elements like you do in Photoshop without using any images. Prior to CSS3, sliced images are used for creating the shadows around the elements that was quite annoying.
The following section will describe you how to apply the shadow on text and elements.

• CSS3 box-shadow Property

The box-shadow property can be used to add shadow to the element's boxes. You can even apply more than one shadow effect using a comma-separated list of shadows. The basic syntax of creating a box shadow can be given with:
box-shadow: offset-x offset-y blur-radius color;
The components of the box-shadow property have the following meaning:

- offset-x — Sets the horizontal offset of the shadow.
- offset-y — Sets the vertical offset of the shadow.
- blur-radius — Sets the blur radius. The larger the value, the bigger the blur and more the shadow's edge will be blurred. Negative values are not allowed.
- color — Sets the color of the shadow. If the color value is omitted or not specified, it takes the value of the color property.

See the CSS3 box-shadow property to learn more about the other possible values.

```
.box{ width: 200px; height: 150px; background: #ccc; box-shadow: 5px 5px 10px #999;
}
```

Note: When adding the box-shadow, if the value for the blur-radius component is not specified, or set to zero (0), the edges of the shadow will be sharp.

Similarly, you can add the multiple box shadow using a comma-separated list:

```
.box{ width: 200px; height: 150px; background: #000; box-shadow: 5px 5px 10px red,
10px 10px 20px yellow; }
```

• CSS3 text-shadow Property

You can use the text-shadow property to apply the shadow effects on text. You can also apply multiple shadows to text using the same notation as box-shadow.

```
h1 { text-shadow: 5px 5px 10px #666; } h2 { text-shadow: 5px 5px 10px red, 10px 10px 20px yellow; }
```

7. CSS3 2D Transforms

With CSS3 2D transform feature you can perform basic transform manipulations such as move, rotate, scale and skew on elements in a two-dimensional space.

A transformed element doesn't affect the surrounding elements, but can overlap them, just like the absolutely positioned elements. However, the transformed element still takes space in the layout at its default (un-transformed) location.

• Using CSS Transform and Transform Functions

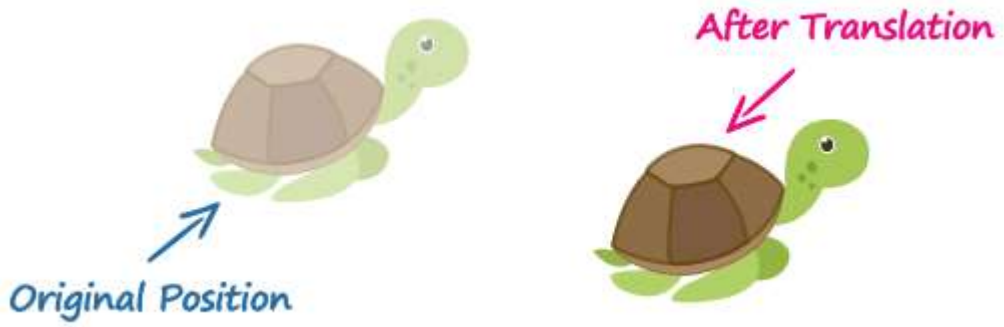
The CSS3 transform property uses the transform functions to manipulate the coordinate system used by an element in order to apply the transformation effect.
The following section describes these transform functions:

• The translate() Function

Moves the element from its current position to a new position along the X and Y axes. This can be written as translate(tx, ty). If ty isn't specified, its value is assumed to be zero.

```
img { -webkit-transform: translate(200px, 50px); /* Chrome, Safari, Opera */ -moz-transform: translate(200px, 50px); /* Firefox */

 -ms-transform: translate(200px, 50px); /* IE 9 */

transform: translate(200px, 50px); /* Standard syntax */ }
```

The function translate(200px, 50px) moves the image 200 pixels horizontally along the positive x-axis, and 50 pixels vertically along the positive y-axis.

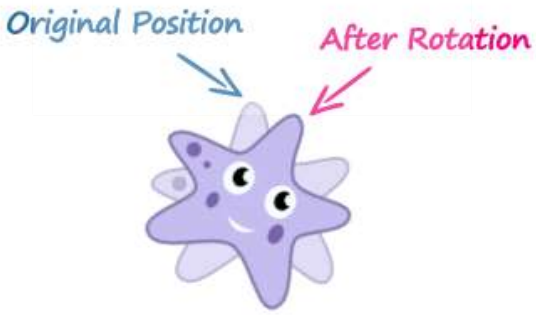


- ## The rotate() Function

The rotate() function rotates the element around its origin (as specified by the transform-origin property) by the specified angle. This can be written as rotate(a).

```
img { -webkit-transform: rotate(30deg); /* Chrome, Safari, Opera */

-moz-transform: rotate(30deg); /* Firefox */

-ms-transform: rotate(30deg); /* IE 9 */

transform: rotate(30deg); /* Standard syntax */ }
```

The function rotate(30deg) rotates the image in clockwise direction about its origin by the angle 30 degrees. You can use negative values to rotate the element counter-clockwise.

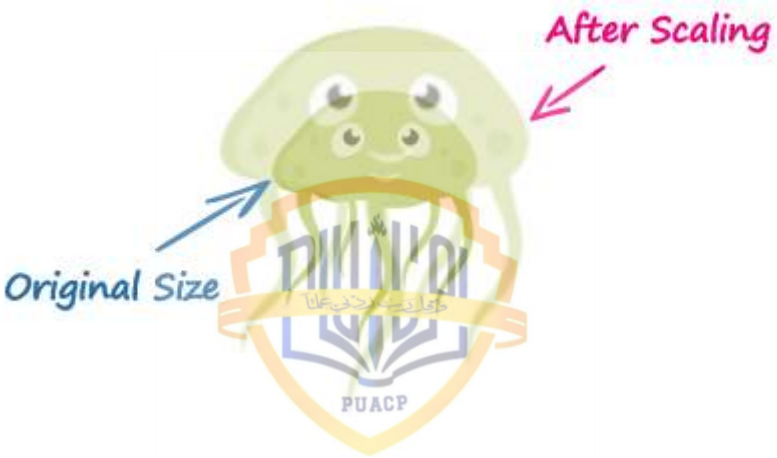


The scale() Function

The scale() function increases or decreases the size of the element. It can be written as scale(sx, sy). If sy isn't specified, it is assumed to be equal to sx.

```
img { -webkit-transform: scale(1.5); /* Chrome, Safari, Opera */

 -moz-transform: scale(1.5); /* Firefox */

-ms-transform: scale(1.5); /* IE 9 */

 transform: scale(1.5); /* Modern Browsers */ }
```

The function scale(1.5) proportionally scale the width and height of the image 1.5 times to its original size. The function scale(1) or scale(1, 1) has no effect on the element.

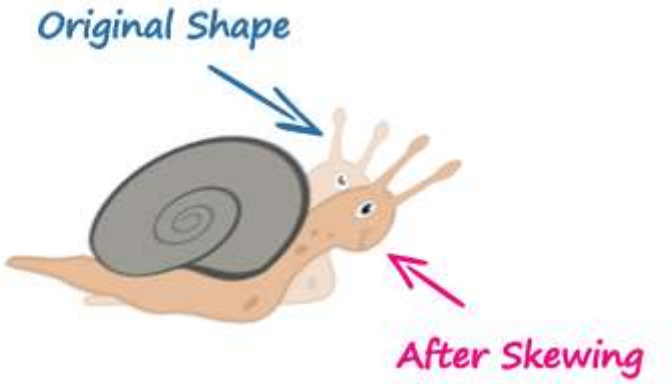


The skew() Function

The skew() function skews the element along the X and Y axes by the specified angles. It can be written as skew(ax, ay). If ay isn't specified, its value is assumed to be zero.

```
img { -webkit-transform: skew(-40deg, 15deg); /* Chrome, Safari, Opera */ -moz-transform: skew(-40deg, 15deg); /* Firefox */

 -ms-transform: skew(-40deg, 15deg); /* IE 9 */

transform: skew(-40deg, 15deg); /* Modern Browsers */ }
```

The function skew(-40deg, 15deg) skews the element -40 degree horizontally along the x-axis, and 15 degree vertically along the y-axis.



The matrix() Function

The matrix() function can perform all of the 2D transformations such as translate, rotate, scale, and skew at once. It takes six parameters in the form of a matrix which can be written as matrix(a, b, c, d, e, f). The following section will show you how each of the 2D transformation functions can be represented using the matrix().

- translate(tx, ty) = matrix(1, 0, 0, 1, tx, ty); — where tx and ty are the horizontal and vertical translation values.
- rotate(a) = matrix(cos(a), sin(a), -sin(a), cos(a), 0, 0); — where a is the value in deg. You can swap the sin(a) and -sin(a) values to reverse the rotation. The maximum rotation you could perform is 360 degrees.
- scale(sx, sy) = matrix(sx, 0, 0, sy, 0 ,0); — where sx and sy are the horizontal and vertical scaling values.
- skew(ax, ay) = matrix(1, tan(ay), tan(ay), 1, 0 ,0); — where ax and ay are the horizontal and vertical values in deg.

Here is an example of performing the 2D transformation using the matrix() function.

```
img { -webkit-transform: matrix(0, -1, 1, 0, 200px, 50px); /* Chrome, Safari, Opera */ -moz-transform: matrix(0, -1, 1, 0, 200px, 50px); /* Firefox */

-ms-transform: matrix(0, -1, 1, 0, 200px, 50px); /* IE 9 */

 transform: matrix(0, -1, 1, 0, 200px, 50px); /* Standard syntax */ }
```

However, when performing more than one transformation at once, it is more convenient to use the individual transformation function and list them in order, like this:

```
img { -webkit-transform: translate(200px, 50px) rotate(180deg) scale(1.5) skew(0, 30deg); /* Chrome, Safari, Opera */ -moz-transform: translate(200px, 50px) rotate(180deg) scale(1.5) skew(0, 30deg); /* Firefox */ -ms-transform: translate(200px, 50px) rotate(180deg) scale(1.5) skew(0, 30deg); /* IE 9 */ transform: translate(200px, 50px) rotate(180deg) scale(1.5) skew(0, 30deg); /* Standard syntax */ }
```

2D Transform Functions

The following table provides a brief overview of all the 2D transformation functions.

Function	Description
translate(tx,ty)	Moves the element by the given amount along the X and Y-axis.
translateX(tx)	Moves the the element by the given amount along the X-axis.
translateY(ty)	Moves the the element by the given amount along the Y-axis.
rotate(a)	Rotates the element by the specified angle around the origin of the element, as defined by the transform-origin property.
scale(sx,sy)	Scale the width and height of the element up or down by the given amount. The function scale(1,1) has no effect.
scaleX(sx)	Scale the width of the element up or down by the given amount.
scaleY(sy)	Scale the height of the element up or down by the given amount.
skew(ax,ay)	Skews the element by the given angle along the X and Y-axis.
skewX(ax)	Skews the element by the given angle along the X-axis.

skewY(ay)	Skews the element by the given angle along the Y-axis.
matrix(n,n,n,n,n,n)	Specifies a 2D transformation in the form of a transformation matrix comprised of the six values.

8. CSS3 3D Transforms

With CSS3 3D transform feature you can perform basic transform manipulations such as move, rotate, scale and skew on elements in a three-dimensional space.
A transformed element doesn't affect the surrounding elements, but can overlap them, just like the absolutely positioned elements. However, the transformed element still takes space in the layout at its default (un-transformed) location.

• Using CSS Transform and Transform Functions

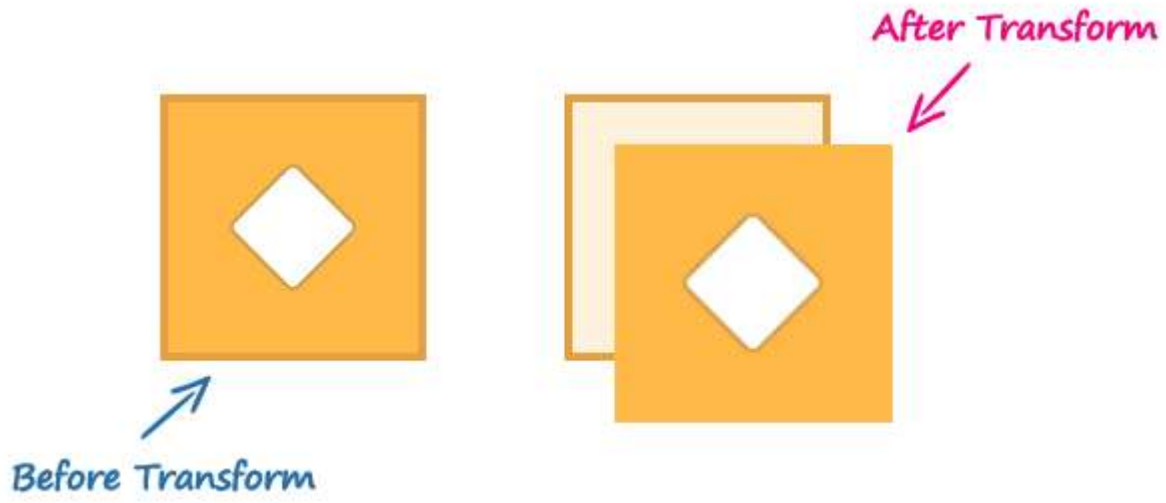
The CSS3 transform property uses the transform functions to manipulate the coordinate system used by an element in order to apply the transformation effect.
The following section describes the 3D transform functions:

• The translate3d() Function

Moves the element from its current position to a new position along the X, Y and Z-axis. This can be written as translate(tx, ty, tz). Percentage values are not allowed for third translation-value parameter (i.e. tz).

```
.container { width: 125px; height: 125px; perspective: 500px; border: 4px solid #e5a043; background: #fff2dd; } .transformed { -webkit-transform: translate3d(25px, 25px, 50px); /* Chrome, Safari, Opera */ transform: translate3d(25px, 25px, 50px); /* Standard syntax */ }
```

The function translate3d(25px, 25px, 50px) moves the image 25 pixels along the positive X and Y-axis, and the 50 pixels along the positive Z-axis.



The 3D transform however uses the three-dimensional coordinate system, but movement along the Z-direction is not always noticeable because the elements exist on a two-dimensional plane (a flat surface) and have no depth.

The perspective and perspective-origin CSS properties can be used to add a feeling of depth to a scene by making the elements higher on the Z-axis i.e. closer to the viewer appear larger, and those further away to the viewer appear smaller.

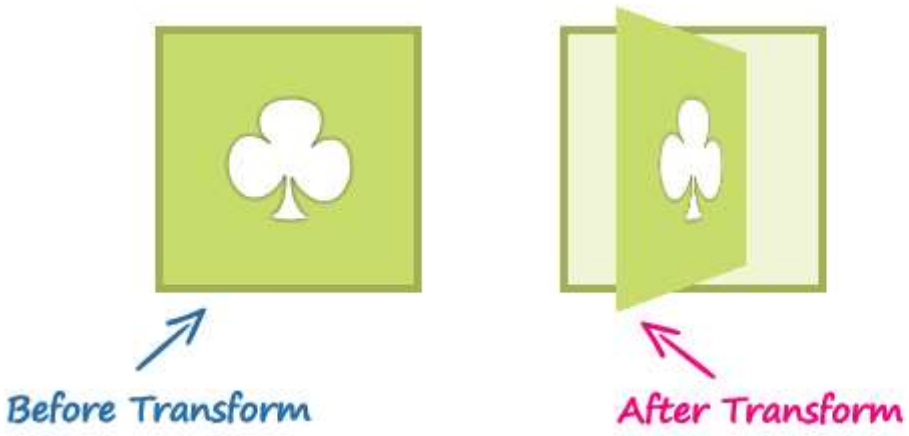
Note: If you apply 3D transformation to an element without setting the perspective the resulting effect will not appear as three-dimensional.

- ## The rotate3d() Function

The rotate3d() function rotates the element in 3D space by the specified angle around the [x,y,z] direction vector. This can be written as rotate(vx, vy, vz, angle).

```
.container{ width: 125px; height: 125px; perspective: 300px; border: 4px solid #a2b058;
background: #f0f5d8; } .transformed { -webkit-transform: rotate3d(0, 1, 0, 60deg); /*
Chrome, Safari, Opera */ transform: rotate3d(0, 1, 0, 60deg); /* Standard syntax */ }
```

The function rotate3d(0, 1, 0, 60deg) rotates the image along the Y-axis by the angle 60 degrees. You can use negative values to rotate the element in opposite direction.

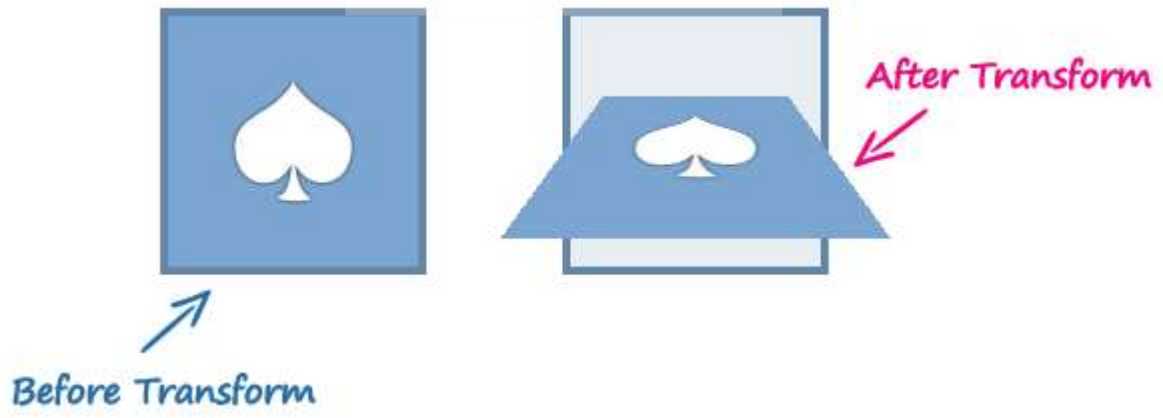


- ## The scale3d() Function

The scale3d() function changes the size of an element. It can be written as scale(sx, sy, sz). The effect of this function is not evident unless you use it in combination with other transform functions such as rotate and the perspective, as shown in the example below.

```
.container{ width: 125px; height: 125px; perspective: 300px; border: 4px solid #6486ab; background: #e9eef3; } .transformed { -webkit-transform: scale3d(1, 1, 2) rotate3d(1, 0, 0, 60deg); /* Chrome, Safari, Opera */ transform: scale3d(1, 1, 2) rotate3d(1, 0, 0, 60deg); /* Standard syntax */ }
```

The function scale3d(1, 1, 2) scales the elements along the Z-axis and the function rotate3d(1, 0, 0, 60deg) rotates the image along the X-axis by the angle 60 degrees.



The matrix3d() Function

The matrix3d() function can perform all of the 3D transformations such as translate, rotate, and scale at once. It takes 16 parameters in the form of a 4×4 transformation matrix.
Here is an example of performing the 3D transformation using the matrix3d() function.

```
.container{ width: 125px; height: 125px; perspective: 300px; border: 4px solid #d14e46; background: #fddddb; } .transformed { -webkit-transform: matrix3d(0.359127, -0.469472, 0.806613, 0, 0.190951, 0.882948, 0.428884, 0, -0.913545, 0, 0.406737, 0, 0, 0, 0, 1); /* Chrome, Safari, Opera */ transform: matrix3d(0.359127, -0.469472, 0.806613, 0, 0.190951, 0.882948, 0.428884, 0, -0.913545, 0, 0.406737, 0, 0, 0, 0, 1); /* Standard syntax */ }
```

However, when performing more than one transformation at once, it is more convenient to use the individual transformation function and list them in order, like this:

```
.container{ width: 125px; height: 125px; perspective: 300px; border: 4px solid #a2b058; background: #f0f5d8; } .transformed { -webkit-transform: translate3d(0, 0, 60px) rotate3d(0, 1, 0, -60deg) scale3d(1, 1, 2); /* Chrome, Safari, Opera */ transform: translate3d(0, 0, 60px) rotate3d(0, 1, 0, -60deg) scale3d(1, 1, 2); /* Standard syntax */ }
```

3D Transform Functions

The following table provides a brief overview of all the 3D transformation functions.

Function	Description
translate3d(tx,ty,tz)	Moves the element by the given amount along the X, Y & Z-axis.

translateX(tx)	Moves the element by the given amount along the X-axis.
translateY(ty)	Moves the element by the given amount along the Y-axis.
translateZ(tz)	Moves the element by the given amount along the Z-axis.
rotate3d(x,y,z, a)	Rotates the element in 3D space by the angle specified in the last parameter, around the [x,y,z] direction vector.
rotateX(a)	Rotates the element by the given angle around the X-axis.
rotateY(a)	Rotates the element by the given angle around the Y-axis.
rotateZ(a)	Rotates the element by the given angle around the Z-axis.
scale3d(sx,sy,sz)	Scales the element by the given amount along the X, Y and Z-axis. The function scale3d(1,1,1) has no effect.
scaleX(sx)	Scales the element along the X-axis.
scaleY(sy)	Scales the element along the Y-axis.

scaleZ(sz)	Scales the element along the Z-axis.
matrix3d(n,n,n,n,n, n, n,n,n,n,n,n,n,n,n,n)	Specifies a 3D transformation in the form of a 4×4 transformation matrix of 16 values.
perspective(length)	Defines a perspective view for a 3D transformed element. In general, as the value of this function increases, the element will appear further away from the viewer.

9. CSS3 Transitions

Normally when the value of a CSS property changes, the rendered result is instantly updated. A common example is changing the background color of a button on mouse hover. In a normal scenario the background color of the button is changes immediately from the old property value to the new property value when you place the cursor over the button.
CSS3 introduces a new transition feature that allows you to animate a property from the old value to the new value smoothly over time. The following example will show you how to animate the background-color of an HTML button on mouse hover.

```
button { background: #fd7c2a; /* For Safari 3.0+ */ -webkit-transition-property: background; -webkit-transition-duration: 2s; /* Standard syntax */ transition-property: background; transition-duration: 2s; } button:hover { background: #3cc16e; }
```

To make the transition occur, you must specify at least two things — the name of the CSS property to which you want to apply the transition effect using the transition-property CSS property, and the duration of the transition effect (greater than 0) using the transition-duration CSS property. However, all the other transition properties are optional, as their default values don't prevent a transition from happening.
Note: Not all CSS properties are animatable. In general, any CSS property that accepts values that are numbers, lengths, percentages, or colors is animatable.

Performing Multiple Transitions

Each of the transition properties can take more than one value, separated by commas, which provides an easy way to define multiple transitions at once with different settings.

```
button { background: #fd7c2a; border: 3px solid #dc5801; /* For Safari 3.0+ */ -webkit-transition-property: background, border; -webkit-transition-duration: 1s, 2s; /* Standard syntax */ transition-property: background, border; transition-duration: 1s, 2s; } button:hover { background: #3cc16e; border-color: #288049; }
```

Transition Shorthand Property

There are many properties to consider when applying the transitions. However, it is also possible to specify all the transition properties in one single property to shorten the code.

The transition property is a shorthand property for setting all the individual transition properties (i.e., transition-property, transition-duration, transition-timing-function, and transition-delay) at once in the listed order.

It's important to stick to this order for the values, when using this property.

```
button { background: #fd7c2a; -webkit-transition: background 2s ease-in 0s; /* For Safari 3.0+ */ transition: background 2s ease-in 0s; /* Standard syntax */ } button:hover { background: #3cc16e; }
```

Note: If any value is missing or not specified, the default value for that property will be used instead. That means, if the value for transition-duration property is missing, no transition will occur, because its default value is 0.

CSS3 Transition Properties

The following table provides a brief overview of all the transition properties:

Property	Description
transition	A shorthand property for setting all the four individual transition properties in a single declaration.
transition-delay	Specifies when the transition will start.
transition-duration	Specifies the number of seconds or milliseconds a transition animation should take to complete.
transition-property	Specifies the names of the CSS properties to which a transition effect should be applied.
transition-timing-function	Specifies how the intermediate values of the CSS properties being affected by a transition will be calculated.

10. CSS3 Animations

In the previous chapter you've seen how to do simple animations like animating a property from one value to another via CSS3 transitions feature. However, the CSS3 transitions provide little control on how the animation progresses over time.

The CSS3 animations take it a step further with keyframe-based animations that allow you to specify the changes in CSS properties over time as a set of keyframes, like flash animations. Creating CSS animations is a two step process, as shown in the example below:

- The first step of building a CSS animation is to defining individual keyframes and naming an animation with a keyframes declaration.
- The second step is referencing the keyframes by name using the animation-name property as well as adding animation-duration and other optional animation properties to control the animation's behavior.

However, it is not necessary to define the keyframes rules before referencing or applying it. The following example will show you how to animate a box horizontally from one position to another using the CSS3 animation feature.

```
.box

{ width: 50px;

 height: 50px;

background: red;

 position: relative;

 /* Chrome, Safari, Opera */

-webkit-animation-name: moveit;

-webkit-animation-duration: 2s;

 /* Standard syntax */

animation-name: moveit; animation-duration: 2s;

 } /* Chrome, Safari, Opera */

@-webkit-keyframes moveit

{

from {left: 0;}
```

```
to {left: 50%;}

} /* Standard syntax */

@keyframes moveit {

from {left: 0;}

to {left: 50%;}

}
```

You must specify at least two properties animation-name and the animation-duration (greater than 0), to make the animation occur. However, all the other animation properties are optional, as their default values don't prevent an animation from happening.

Note: Not all CSS properties are animatable. In general, any CSS property that accepts values that are numbers, lengths, percentages, or colors is animatable.

- ## Defining Keyframes

Keyframes are used to specify the values for the animating properties at various stages of the animation. Keyframes are specified using a specialized CSS at-rule — @keyframes. The keyframe selector for a keyframe style rule starts with a percentage (%) or the keywords from (same as 0%) or to (same as 100%). The selector is used to specify where a keyframe is constructed along the duration of the animation.
Percentages represent a percentage of the animation duration, 0% represents the starting point of the animation, 100% represents the end point, 50% represents the midpoint and so on. That means, a 50% keyframe in a 2s animation would be 1s into an animation.

```
.box { width: 50px;

 height: 50px;

margin: 100px;

background: red;

 position: relative;
```

```
left: 0; /* Chrome, Safari, Opera */
-webkit-animation-name: shakeit;
 -webkit-animation-duration: 2s; /* Standard syntax */
animation-name: shakeit; animation-duration: 2s; }
/* Chrome, Safari, Opera */
@-webkit-keyframes shakeit
{
12.5% {left: -50px;}
 25% {left: 50px;}
 37.5% {left: -25px;} 50% {left: 25px;}
62.5% {left: -10px;} 7
5% {left: 10px;} }
/* Standard syntax */
@keyframes shakeit
{
12.5% {left: -50px;}
 25% {left: 50px;}
37.5% {left: -25px;}
 50% {left: 25px;}
 62.5% {left: -10px;}
75% {left: 10px;}
}
```

Animation Shorthand Property

There are many properties to consider when creating the animations. However, it is also possible to specify all the animations properties in one single property to shorten the code.
The animation property is a shorthand property for setting all the individual animation properties at once in the listed order. For example:

```
.box

{ width: 50px;

height: 50px;

 background: red;

position: relative;

 /* Chrome, Safari, Opera */

-webkit-animation: repeatit 2s linear 0s infinite alternate;

 /* Standard syntax */

 animation: repeatit 2s linear 0s infinite alternate; }

/* Chrome, Safari, Opera */

@-webkit-keyframes repeatit { from {left: 0;} to {left: 50%;} }

 /* Standard syntax */

@keyframes repeatit

 { from {left: 0;} to {left: 50%;} }
```

Note: If any value is missing or not specified, the default value for that property will be used instead. That means, if the value for animation-duration property is missing, no transition will occur, because its default value is 0.

• CSS3 Animation Properties.

The following table provides a brief overview of all the animation-related properties.

Property	Description
`animation`	A shorthand property for setting all the animation properties in single declaration.
`animation-name`	Specifies the name of @keyframes defined animations that should be applied to the selected element.
`animation-duration`	Specifies how many seconds or milliseconds that an animation takes to complete one cycle of the animation.
`animation-timing-function`	Specifies how the animation will progress over the duration of each cycle i.e. the easing functions.

animation-delay	Specifies when the animation will start.
animation-iteration-count	Specifies the number of times an animation cycle should be played before stopping.
animation-direction	Specifies whether or not the animation should play in reverse on alternate cycles.
animation-fill-mode	Specifies how a CSS animation should apply styles to its target before and after it is executing.
animation-play-state	Specifies whether the animation is running or paused.
@keyframes	Specifies the values for the animating properties at various points during the animation.

11. CSS3 Multi-Column Layouts

• Creating Multi-Column Layouts

The CSS3 has introduced the multi-column layout module for creating multiple column layouts in an easy and efficient way. Now you can create layouts like you see in magazines and newspapers without using the floating boxes. Here is a simple example of splitting some text into three columns using the CSS3 multiple column layout feature.

```
p { -webkit-column-count: 3; /* Chrome, Safari, Opera */

 -moz-column-count: 3; /* Firefox */

column-count: 3; /* Standard syntax */ }
```

• Setting Column Count or Width

The CSS properties column-count and column-width control whether and how many columns will appear. The column-count property sets the number of columns inside the multicol element, whereas the column-width property sets the desired width of the columns.

```
p { -webkit-column-width: 150px; /* Chrome, Safari, Opera */ -moz-column-width:
150px; /* Firefox */ column-width: 150px; /* Standard syntax */ }
```

Note: The column-width describes the optimal width of the column. However, the actual column width may be wider or narrower according to the space available. This property should not be used with the column-count property.

• Setting Column Gap

You can control the gap between columns using the column-gap property. The same gap is applied between each column. The default gap is normal which is equivalent to 1em.

```
p { /* Chrome, Safari, Opera */

 -webkit-column-count: 3;
```

```
-webkit-column-gap: 100px;

 /* Firefox */

-moz-column-count: 3;

 -moz-column-gap: 100px;

/* Standard syntax */

column-count: 3;

column-gap: 100px; }
```

• Setting Column Rules

You can also add a line between the columns i.e. the rule using the column-rule property. It is a shorthand property for setting width, style, and color of the rule in a single declaration. The column-rule property takes the same values as border and outline.

```
p { /* Chrome, Safari, Opera */ -webkit-column-count: 3; -webkit-column-gap: 100px; -webkit-column-rule: 2px solid red; /* Firefox */ -moz-column-count: 3; -moz-column-gap: 100px; -moz-column-rule: 2px solid red; /* Standard syntax */ column-count: 3; column-gap: 100px; column-rule: 2px solid red; }
```

Note: The width of column rule does not affect the width of column boxes, but if a column rule is wider than the gap, the adjacent column boxes will overlap the rule.

• CSS3 Multiple Columns Properties

The following table provides a brief overview of all the multiple columns properties:

Property	Description

column-count	Specifies the number of columns inside a multi-column element.
column-fill	Specifies how content is spread across columns.
column-gap	Specifies the gap between the columns.
column-rule	Specifies a straight line or rule, to be drawn between each column.
column-rule-color	Specifies the color of the rule between columns.
column-rule-style	Specifies the style of the rule between columns.
column-rule-width	Specifies the width of the rule between columns.
column-span	Specifies how many columns an element spans across.
column-width	Specifies the optimal width of the columns.
columns	A shorthand property for setting both the column-width and the column-count properties at the same time.

12. CSS3 Box Sizing

- **Redefining Box Width with Box-Sizing**

By default, the actual width or height of an element's box visible/rendered on a web page depends on its width or height, padding and border property. For example, if you apply some padding and border on a element with 100% width the horizontal scrollbar will appear, as you can see in the example below.

```
.box { width: 100%; padding: 20px; border: 5px solid #f08080; }
```

This is very common problem that web designers are facing for a long time. But, CSS3 introduces the box-sizing property to solve this problem and make the CSS layouts much more simple and intuitive. The box-sizing property alter the default CSS box model in such a way that, any padding or border specified on the element is laid out and drawn inside of the content area, so that the rendered width and height of the element is equal to the specified CSS width and height properties.

```
.box { width: 100%; padding: 20px; border: 5px solid #f08080; box-sizing: border-box;
}
```

If you see the output of this example, you will find the scroll bar has disappeared.

Note: When using the CSS box-sizing property the resulting width and height of the content area are calculated by subtracting the border and padding widths of the respective sides from the specified width and height properties.

- ## Creating Layouts with Box-Sizing

With the CSS box-sizing property creating the multiple columns layout using percentages becomes very simple. Now you don't have to worry about the final size of the element's box while adding the padding or border on them.
The following example will create a two column layout where each column has equal width and are placed side-by-side using the float property.

```
.box { width: 50%; padding: 20px; background: #f2f2f2; float: left; box-sizing: border-box; }
```

Similarly, you create more complex layouts using this simple technique.

```
.box { width: 30%; padding: 20px; margin-left: 5%; background: #f2f2f2; float: left; box-sizing: border-box; } .box:first-child { margin-left: 0; }
```

13. CSS3 Flexible Box Layouts

Flexible box, commonly referred to as flexbox, is a new layout model introduced in CSS3 for creating the flexible user interface design with multiple rows and columns without using the percentage or fixed length values. The CSS3 flex layout model provides a simple and powerful

mechanism for handling the distributing of space and alignment of content automatically through stylesheet without interfering the actual markup.
The following example demonstrate how to create a three column layout where each column has equal width and height using the flex layout model.

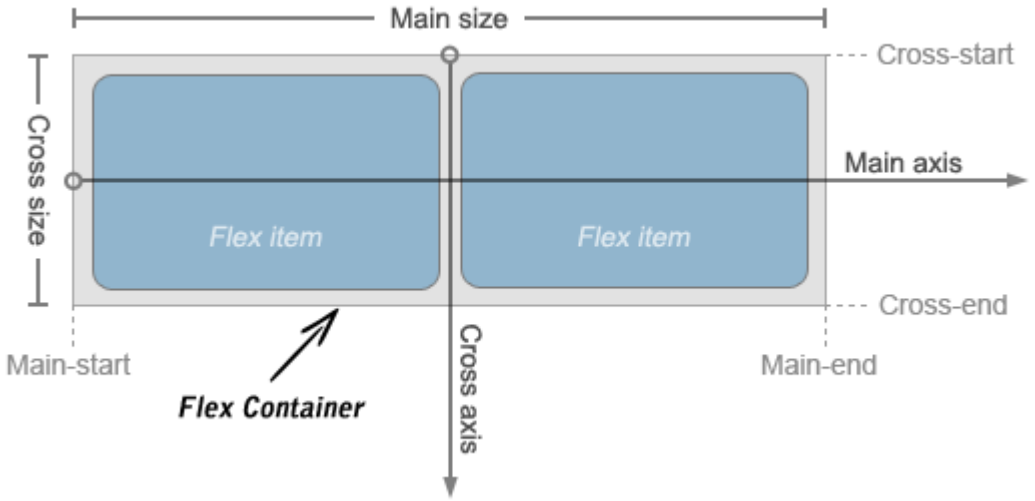
```
<!DOCTYPE html>

<html lang="en">

<head> <meta charset="utf-8">

<title>CSS3 Three Equal Flex Column</title>

<style>

.flex-container {

width: 80%;

min-height: 300px;

display: -webkit-flex;

/* Safari */

display: flex;

/* Standard syntax */

border: 1px solid #808080; }

.flex-container div

{ background: #dbdfe5; -webkit-flex: 1;

/* Safari */
```

```
-ms-flex: 1;
 /* IE 10 */
flex: 1;
 /* Standard syntax */ }
</style> </head> <body> <div class="flex-container">
<div class="item1">Item 1</div>
<div class="item2">Item 2</div>
<div class="item3">Item 3</div>
</div>
 </body>
</html>
```

If you notice the above example code carefully you'll find we didn't specify any width on the inner of the .flex-container, but you can see in the output, every column has width which is exactly equal to the one third of the parent .flex-container element.

- ## How Flex Layout Works

Flexbox consists of flex containers and flex items. A flex container can be created by setting the display property of an element to either flex (generate block-level flex container) or inline-flex (generate an inline flex container similar to inline-block). All child elements of flex container automatically become flex items and are laid out using the flex layout model. The float, clear, and vertical-align properties have no effect on flex items.
Flex items are positioned inside a flex container along a flex line controlled by the flex-direction property. By default, there is only one flex line per flex container which has the same orientation as the inline axis of the current writing mode or text direction. The following illustration will help you to understand the flex layout terminology.



• Controlling Flow inside Flex Container

In the standard CSS box model, the elements normally displayed in the order as they appear in the underlying HTML markup. Flex layout lets you control the direction of the flow inside a flex container in such a way that the elements can be laid out in any flow direction leftwards, rightwards, downwards, or even upwards.
The flow of the flex items in a flex container can be specified using the flex-direction property. The default value of this property is row which is same as the document's current writing mode or text direction e.g. left-to-right in English language.

```
.flex-container { width: 80%; min-height: 300px; /* Safari */ display: -webkit-flex; -webkit-flex-direction: row-reverse; /* Standard syntax */ display: flex; flex-direction: row-reverse; border: 1px solid #666; }
```

Similary, you can disply the flex items inside a flex container in columns instead of row using the value column for the flex-direction property, like this:

```
.flex-container { width: 80%; min-height: 300px; /* Safari */ display: -webkit-flex; -webkit-flex-direction: column; /* Standard syntax */ display: flex; flex-direction: column; }
```

Controlling the Dimensions of Flex Items

The most important aspect of the flex layout is the ability of flex items to alter their width or height to fill the available space. This is achieved with the flex property. It is shorthand property for flex-grow, flex-shrink and flex-basis properties.
A flex container distributes the free space to its items proportional to their flex grow factor, or shrinks them to prevent overflow proportional to their flex shrink factor.

```
.flex-container { width: 80%; min-height: 300px; display: -webkit-flex; /* Safari */
display: flex; /* Standard syntax */ } .flex-container div { padding: 10px; background:
#dbdfe5; } .item1, .item3 { -webkit-flex: 1; /* Safari */ flex: 1; /* Standard syntax */ }
.item2 { -webkit-flex: 2; /* Safari */ flex: 2; /* Standard syntax */ }
```

In the above example, the first and third flex items will occupy $^1/_4$ i.e. $^1/_{(1+1+2)}$ of the free space each, whereas the second flex item will occupy $^1/_2$ i.e. $^2/_{(1+1+2)}$ of the free space. Similarly, you can create other flexible layouts using this simple technique.

Note: It is strongly recommended to use the shorthand rather than the individual flex properties because it resets unspecified components correctly.

Aligning Flex Items within Flex Container

There are four properties justify-content, align-content, align-items and align-self which are designed to control the alignment of flex items within flex container. The first three of them apply to flex containers whereas the last one applies to the individual flex items.

Align Flex Items along Main Axis

Flex items can be aligned along the main axis (i.e. in the horizontal direction) of the flex container using the justify-content property. It is typically used when the flex items do not use all the space available along the main axis.
The justify-content property accepts the following values:

- flex-start — Default value. Flex items are placed at the start of the main axis.
- flex-end — Flex items are placed at the end of the main axis.
- center — Flex items are placed at the center of the main axis with equal amounts of free space at both ends. If the leftover free-space is negative i.e. if the items overflow, then the flex items will overflow equally in both directions.
- space-between — Flex items are distributed evenly along the main axis, where the first item placed at the main-start edge and the last one placed at the main-end. If items overflow or there's only one item, this value is equal to flex-start.

- space-around — Flex items are distributed evenly with half-size spaces on either end. If they overflow or there's only one item, this value is identical to center.

The following example will show you the effect of the different values for justify-content property on a multiple-column flex container having fixed width.

```
.flex-container { width: 500px; min-height: 300px; border: 1px solid #666; /* Safari */
display: -webkit-flex; -webkit-justify-content: space-around; /* Standard syntax */
display: flex; justify-content: space-around; } .item1 { width: 75px; background:
#e84d51; } .item2 { width: 125px; background: #7ed636; } .item3 { width: 175px;
background: #2f97ff; }
```

Align Flex Items along Cross Axis

Flex items can be aligned along the cross axis (i.e. in the perpendicular direction) of the flex container using the align-items or align-self property. However, where the align-items is applied to the flex container, the align-self property is applied to the individual flex items, and overrides align-items. Both properties accept the following values:

- flex-start — Flex items are placed at the start of the cross axis.
- flex-end — Flex items are placed at the end of the cross axis.
- center — Flex items are placed at the center of the cross axis with equal amounts of free space at both ends. If the leftover free-space is negative i.e. if the items overflow, then the flex items will overflow equally in both directions.
- baseline — The text baseline (or inline axis) of each flex item is aligned with the baseline of the flex item with the largest font-size.
- stretch — The flex item stretches to fill the current row or column unless prevented by the minimum and maximum width or height. Default value for align-items property.

The following example will show you the effect of the different values for align-items property on a multiple-column flex container having fixed height.

```
.flex-container { width: 500px; min-height: 300px; border: 1px solid #666; /* Safari */
display: -webkit-flex; -webkit-align-items: center; /* Standard syntax */ display: flex;
align-items: center; } .item1 { width: 75px; height: 100px; background: #e84d51; }
.item2 { width: 125px; height: 200px; background: #7ed636; } .item3 { width: 175px;
height: 150px; background: #2f97ff; }
```

You can also distribute free space on the cross axis of a multiple-row or multiple-column flex container. The property align-content is used to align the flex container's lines, for example, rows within the multiple-row flex container when there is extra space in the cross-axis, similar to how justify-content aligns individual items within the main-axis.
The align-content property accepts the same values as justify-content, but applies them to the cross axis rather than the main axis. It also accepts one more value:

- stretch The free space is split equally between all rows or columns increasing their cross size. If the leftover free-space is negative, this value is identical to flex-start.

The following example will show you the effect of the different values for align-content property on a multiple-row flex container having fixed height.

```
.flex-container

{ width: 500px;

min-height: 300px;

margin: 0 auto;

font-size: 32px;

 border: 1px solid #666;

/* Safari */

display: -webkit-flex;

 -webkit-flex-flow: row wrap;

 -webkit-align-content: space-around;

 /* Standard syntax */

display: flex;

flex-flow: row wrap;

 align-content: space-around; }
```

```
.flex-container div { width: 150px;

height: 100px;

background: #dbdfe5; }
```

• Reordering Individual Flex Items

In addition to altering the flow within a flex container, you can also change the order in which individual flex items are displayed using the order property. This property accepts positive or negative integer as value. By default, all flex items are displayed and laid out in the same order as they appear in the HTML markup as the default value of order is 0.
The following example will show you how to control the order of an individual flex item.

```
.flex-container

{ width: 500px;

min-height: 300px;

border: 1px solid #666;

display: -webkit-flex;

/* Safari 6.1+ */

display: flex; }

.flex-container div {

padding: 10px;

width: 130px; }

.item1 { background: #e84d51;
```

```
-webkit-order: 2; /* Safari 6.1+ */ order: 2; }

.item2 { background: #7ed636; -webkit-order: 1;

/* Safari 6.1+ */ order: 1; }

.item3 { background: #2f97ff; -webkit-order: -1; /* Safari 6.1+ */ order: -1; }
```

Note: Flex item with the lowest order value laid out first, whereas the item with highest order value laid out at the end. Items with the same order value are displayed in the same order a they appear in the source document.

• Horizontal and Vertical Center Alignment with Flexbox

Normally vertical alignment of a content block involves the use of JavaScript or some ugly CSS tricks. But with flexbox you can easily do this without any adjustments.
The following example will show you how to align a content block vertically and horizontally in the middle easily with the CSS3 flexible box feature.


```
.flex-container

{ width: 500px;

 min-height: 300px;

 border: 1px solid #666;

display: -webkit-flex;

/* Safari 6.1+ */

 display: flex; /* Standard syntax */ }

.item { width: 300px;

padding: 25px;
```

```
margin: auto;

background: #f0e68c; }
```

• Enable Wrapping of Flex Items

By default, flex containers display only a single row or column of flex items. However, you can use the flex-wrap property on the flex container to control whether its flex items will wrap into multiple lines or not, if there is not sufficient space for them on one flex line.
The flex-wrap property accept the following values:

- nowrap — Default value. The flex items are placed in a single line. It may cause overflow, if there is not enough space on the flex line.
- wrap — The flex container breaks its flex items across multiple lines, similar to how text is broken onto a new line when it is too wide to fit on the current line.
- wrap-reverse — The flex items will wrap, if necessary, but in reverse order i.e. the cross-start and cross-end directions are swapped.

The following example will show you how to display the flex items in a single or multiple lines within a flex container using the flex-wrap property.

```
.flex-container

{ width: 500px;

min-height: 300px;

border: 1px solid #666; /* Safari */

display: -webkit-flex;

-webkit-flex-wrap: wrap; /* Standard syntax */

display: flex; flex-wrap: wrap; } .

flex-container div{ width: 130px;

padding: 10px;
```

background: **#dbdfe5**; }

Note: You can also use the shorthand CSS property flex-flow for setting both the flex-direction and flex-wrap in a single declaration. It accepts the same values as the individual properties and the values can be in any order.

14. CSS3 Filters

In this chapter we'll discuss about the filter effects introduced in CSS3 that you can use to perform visual effect operations like blur, balancing contrast or brightness, color saturation etc. on graphical elements like an image before it is drawn onto the web page.
The filter effects can be applied to the element using the CSS3 filter property, which accept one or more filter function in the order provided.

```
filter:     blur() | brightness() | contrast() | drop-shadow() | grayscale() | hue-rotate() |
            invert() | opacity() | sepia() | saturate() | url();
```

Warning: The CSS3 filter effects currently not supported in any version of the Internet Explorer. Older versions of IE supported a non-standard filter property for creating effects like opacity, but this feature has been deprecated.

• The Blur Effect

Photoshop like Gaussian blur effect can be applied to an element using the blur() function. This function accepts CSS length value as parameter which defines the blur radius. A larger value will create more blur. If no parameter is provided, then a value 0 is used.

```
img { -webkit-filter: blur(5px); /* Chrome, Safari, Opera */ filter: blur(5px); }
```

— The output of the above example will look something like this:

blur(0)

blur(2px)

blur(5px)

- ## Setting the Image Brightness

The brightness() function can be used to set the brightness of an image. A value of 0% will create an image that is completely black. Whereas, a value of 100% or 1 leaves the images unchanged. Other values are linear multipliers on the effect.
You can also set the brightness higher than the 100% which results in brighter image. If the amount parameter is missing, a value of 1 is used. Negative values are not allowed.

```
img.bright { -webkit-filter: brightness(200%); /* Chrome, Safari, Opera */ filter: brightness(200%); } img.dark { -webkit-filter: brightness(50%); /* Chrome, Safari, Opera */ filter: brightness(50%); }
```

— The output of the above example will look something like this:

brightness(50%)

brightness(100%)

brightness(200%)

Note: The filter functions that take a value expressed with a percent sign (e.g. 75%) also accept the value expressed as decimal (like, 0.75). If the value is invalid, the function returns none and no filter effect will be applied.

• Adjusting the Image Contrast

The contrast() function is used to adjust the contrast on an image. A value of 0% will create an image that is completely black. Whereas, a value of 100% or 1 leaves the image unchanged. Values over 100% are also allowed which provide results with less contrast. If the amount parameter is missing or omitted, a value of 1 is used.

```
img.bright { -webkit-filter: contrast(200%); /* Chrome, Safari, Opera */ filter: contrast(200%); } img.dim { -webkit-filter: contrast(50%); /* Chrome, Safari, Opera */ filter: contrast(50%); }
```

— The output of the above example will look something like this:

contrast(50%)

contrast(100%)

contrast(200%)

• Adding Drop Shadow to Images

You can use the drop-shadow() function to apply the drop shadow effect to the images like Photoshop. This function is similar to the box-shadow property.

```
img { -webkit-filter: drop-shadow(4px 4px 10px orange); /* Chrome, Safari, Opera */ filter: drop-shadow(4px 4px 10px orange); }
```

— The output of the above example will look something like this:

drop-shadow(0)

drop-shadow(2px 2px 4px orange)

drop-shadow(4px 4px 10px orange)

Note: The first and second parameters of the drop-shadow() function specifies the horizontal and vertical offset of the shadow respectively, whereas the third parameter specify the blur radius and the last parameter specifies the shadow color, just like the box-shadow property, with one exception, the 'inset' keyword is not allowed.

Converting an Image to Grayscale

The images can be converted to grayscale using the grayscale() function. A value of 100% is completely grayscale. A value of 0% leaves the image unchanged. Values between 0% and 100% are linear multipliers on the effect. If the amount parameter is missing, a value of 0 is used.

```
img.complete-gray { -webkit-filter: grayscale(100%); /* Chrome, Safari, Opera */ filter: grayscale(100%); } img.partial-gray { -webkit-filter: grayscale(50%); /* Chrome, Safari, Opera */ filter: grayscale(50%); }
```

— The output of the above example will look something like this:

grayscale(0)

grayscale(50%)

grayscale(100%)

- ## Applying Hue Rotation on Image

The hue-rotate() function applies a hue rotation on the image. The passed parameter defines the number of degrees around the color circle the image samples will be adjusted. A value of 0deg leaves the image unchanged. If the 'angle' parameter is missing, a value of 0deg is used. There is no maximum value, the effect of values above 360deg wraps around.

```
img.hue-normal { -webkit-filter: hue-rotate(150deg); /* Chrome, Safari, Opera */ filter: hue-rotate(150deg); } img.hue-wrap { -webkit-filter: hue-rotate(480deg); /* Chrome, Safari, Opera */ filter: hue-rotate(480deg); }
```

— The output of the above example will look something like this:

hue-rotate(0)

hue-rotate(150deg)

hue-rotate(480deg)

The Invert Effect

The invert effect like Photoshop can be applied to an image with the invert() function. A value of 100% or 1 is completely inverted. A value of 0% leaves the input unchanged. Values between 0% and 100% are linear multipliers on the effect. If the 'amount' parameter is missing, a value of 0 is used. Negative values are not allowed.

```
img.partially-inverted { -webkit-filter: invert(80%); /* Chrome, Safari, Opera */ filter: invert(80%); } img.fully-inverted { -webkit-filter: invert(100%); /* Chrome, Safari, Opera */ filter: invert(100%); }
```

— The output of the above example will look something like this:

`invert(0)`

`invert(80%)`

`invert(100%)`

Applying Opacity to Images

The opacity() function can be used to apply transparency to the images. A value of 0% is completely transparent. A value of 100% or 1 leaves the image unchanged. Values between 0% and 100% are linear multipliers on the effect. If the 'amount' parameter is missing, a value of 1 is used. This function is similar to the opacity property.

```
img { -webkit-filter: opacity(80%); /* Chrome, Safari, Opera */ filter: opacity(80%); }
```

— The output of the above example will look something like this:

opacity(100%)

opacity(80%)

opacity(30%)

- ## Applying Sepia Effect to Images

The sepia() function converts the image to sepia. A value of 100% or 1 is completely sepia. A value of 0% leaves the image unchanged. Values between 0% and 100% are linear multipliers on the effect. If the 'amount' parameter is missing, a value of 0 is used.

```
img.complete-sepia { -webkit-filter: sepia(100%); /* Chrome, Safari, Opera */ filter: sepia(100%); } img.partial-sepia { -webkit-filter: sepia(30%); /* Chrome, Safari, Opera */ filter: sepia(30%); }
```

— The output of the above example will look something like this:

sepia(0%)

sepia(30%)

sepia(100%)

Tip: In photographic terms, sepia toning is a specialized treatment to give a black-and-white photograph a warmer tone (reddish-brown) to enhance its archival quality.

• Adjusting the Saturation of Images

The saturate() function can be used to adjust the saturaion of an image. A value of 0% is completely un-saturated. A value of 100% leaves the image unchanged. Other values are linear multipliers on the effect. Values of amount over 100% are also allowed, providing super-saturated results. If the 'amount' parameter is missing, a value of 1 is used.

```
img.un-saturated { -webkit-filter: saturate(0%); /* Chrome, Safari, Opera */ filter: saturate(0%); } img.super-saturated { -webkit-filter: saturate(200%); /* Chrome, Safari, Opera */ filter: saturate(200%); }
```

— The output of the above example will look something like this:

saturate(100%)

saturate(200%)

saturate(0%)

Note: The url() function specifies a filter reference to a specific filter element. For example, url(commonfilters.svg#foo). If the filter reference to an element that didn't exist or the referenced element is not a filter element, then the whole filter chain is ignored. No filter is applied to the element.

15. CSS3 Media Queries

• Media Queries and Responsive Web Design

Media queries allow you to customize the presentation of your web pages for a specific range of devices like mobile phones, tablets, desktops, etc. without any change in markups. A media

query consists of a media type and zero or more expressions that match the type and conditions of a particular media features such as device width or screen resolution.
Since media query is a logical expression it can be resolve to either true or false. The result of the query will be true if the media type specified in the media query matches the type of device the document is being displayed on, as well as all expressions in the media query are satisfied. When a media query is true, the related style sheet or style rules are applied to the target device. Here's a simple example of the media query for standard devices.

```
/* Smartphones (portrait and landscape) ---------- */

@media screen and (min-width: 320px) and (max-width: 480px)

{ /* styles */ } /* Smartphones (portrait) ---------- */
@media screen and (max-width: 320px)

{ /* styles */ } /* Smartphones (landscape) ---------- */
@media screen and (min-width: 321px)

{ /* styles */ }

 /* Tablets, iPads (portrait and landscape) ---------- */
@media screen and (min-width: 768px) and (max-width: 1024px)

{ /* styles */ } /* Tablets, iPads (portrait) ---------- */
@media screen and (min-width: 768px){ /* styles */ } /*
Tablets, iPads (landscape) ---------- */ @media screen and
(min-width: 1024px){ /* styles */ } /* Desktops and laptops --
-------- */ @media screen and (min-width: 1224px){ /* styles
*/ } /* Large screens ---------- */ @media screen and (min-
width: 1824px){ /* styles */ }
```

Tip: Media queries are an excellent way to create responsive layouts. Using media queries you can customize your website differently for users browsing on devices like smart phones or tablets without changing the actual content of the page.

Changing Column Width Based on Screen Size

You can use the CSS media query for changing the web page width and related elements to offer the best viewing experience for the user on different devices.
The following style rules will automatically change the width of the container element based on the screen or viewport size. For example, if the viewport width is less than 768 pixels it will cover the 100% of the viewport width, if it is greater than the 768 pixels but less than the 1024 pixels it will be 750 pixels wide, and so on.

```
.container

 { margin: 0 auto;

 background: #f2f2f2;

 box-sizing: border-box; }

/* Mobile phones (portrait and landscape) ---------- */

@media screen and (max-width: 767px)

{ .container

 { width: 100%; padding: 0 10px; }

 }

/* Tablets and iPads (portrait and landscape) ---------- */
@media screen and (min-width: 768px) and (max-width: 1023px)

{ .container { width: 750px; padding: 0 10px; }

} /* Low resolution desktops and laptops ---------- */

 @media screen and (min-width: 1024px)

 { .container { width: 980px; padding: 0 15px; } }
```

```
/* High resolution desktops and laptops ---------- */

@media screen and (min-width: 1280px)

 { .container

{ width: 1200px; padding: 0 20px; } }
```

Note: You can use the CSS3 box-sizing property on the elements to create more intuitive and flexible layouts with much less effort.

• Changing Layouts Based on Screen Size

You can also use the CSS media query for making your multi-column website layout more adaptable and responsive for devices through little customization.
The following style rule will create a two column layout if the viewport size is greater than or equal to 768 pixels, but if less than that it'll be rendered as one column layout.

```
.column

 { width: 48%;

 padding: 0 15px;

 box-sizing: border-box;

background: #93dcff;

float: left; }

.container

.column:first-child{ margin-right: 4%; }

 @media screen and (max-width: 767px)
```

```
{ .column

{ width: 100%;

 padding: 5px 20px;

 float: none; }

.container .column:first-child{ margin-right: 0; margin-
bottom: 20px; } }
```

16. CSS3 Miscellaneous

• Extending User Interface with CSS3

In this chapter we'll discuss about some interesting user interface related CSS3 properties like resize, outline-offset, etc. that you can use to enhance your web pages.

Resizing Elements

You can make an element resizable horizontally, vertically or on both directions with the CSS3 resize property. However, this property is typically used to remove the default resizable behavior form the <textarea> form control.

```
p, div, textarea { width: 300px; min-height: 100px; overflow: auto; border: 1px solid
black; } p { resize: horizontal; } div { resize: both; } textarea { resize: none; }
```

• Setting Outline Offset

In the previous section you've learnt how to set different properties for outlines like width, color and styles. However, CSS3 offer one more property outline-offset for setting the space between an outline and the border edge of an element. This property can accepts negative value that means you can also place outline inside an element.

```
p, div

{ margin: 50px;
```

```
height: 100px;

background: #000;

outline: 2px solid red; }

p { outline-offset: 10px; }

div { outline-offset: -15px; }
```

TOPIC END